

Tutorial Letter 103/0/2026

Introduction to Programming II COS1512 Assessment 3

Year Module(s): 2026

Name of Department: Computer Science

IMPORTANT INFORMATION

Please register on myUnisa, activate your myLife e-mail account and make sure that you have regular access to the myUnisa module website, MODULE COS1512-2026-Y, as well as your group website.

Note: This is a fully online module. It is, therefore, only available on myUnisa.

CONTENTS

	<i>Page</i>
1. Introduction.....	3
2. Assignment 3 Instructions	3
3. Question 1 [22]	4
4. Question 2 [20]	5
5. Question 3 [28]	6
6. Question 4 [30]	7

1. Introduction

Kindly note the total mark for this assessment is [100], i.e. **Total = 100**. This assessment covers topics studied from all chapters of the prescribed books. The assessment consists of four questions.

2. Assignment 3 Instructions

Assignment Instructions: Workshop Peer- Assessment

Assignment 3 is a **four-phase assessment**, and you must note and act according to the requirements for each phase. The instructions for each phase are as follows:

Phase 1 – The File Submission Phase

- **Complete the questions.** Write the code to answer the question.
- While developing the solution, use comments on the code to indicate the purpose of the functions implemented.
- Answer questions in sequence or given order. The solution should be provided or done in a Word document and converted to PDF for submission. **Copy and paste the code to the document including the screenshot of the output. Both the code and comments and screenshot of the output must be submitted in a PDF file.**
- You must save the PDF document with your student number and the assessment as the name e.g. 12345678_A2.pdf.

File Submission

- The assessment must be uploaded as a single PDF document.

The grading phase will open as soon as the submission phase has closed, you will have one **(1) week to complete the grading phase.**

Phase 2 - Testing

Once phase 1 (submission) is closed, you are required to return to the assessment.

Download at least three (3) of your fellow student's submissions, and follow the instructions provided to see if you can solve the programming problem.

The testing/grading phase will open as soon as the submission phase has closed, **you will have one (1) week to complete the testing/grading phase.**

Phase 3 - Grading

Return to the assessment for **phase 3 and grade the three (3) students' step-by-step instructions, as well as your own, using the rubric that will be provided.** You will thus **grade four (4) assessments in total.**

Your result will be dependent on how you grade yourself and your fellow students, as well as how you are graded. If you grade too high or too low in comparison with the other students who grade the same assessment, your grading will negatively impact your own grade. You thus need to make sure you provide a fair and honest grade.

The testing/grading phase will open as soon as the submission phase has closed, you will have one (1) week to complete the testing/grading phase.

Phase 4

Once the assessment phase is closed, return to your assessment. **Read the feedback and comments provided by your fellow students.** If you have any queries related to your feedback, please send an email to the lecturer detailing why you do not agree with the assessment you received. **You have one (1) week after the close of the grading phase to request moderation of your grade.**

3. Question 1 [22]

Consider the following class:

```
class Computer
{
public:
    Computer();
    Computer(int s, int mm, int hm);
    void display_specs(ostream & out) const;
    int get_speed() const;
    int get_mmemory() const;
    int get_hmemory() const;
private:
    int speed;
    int main_memory;
    int harddisk_memory;
};
```

3.1. Implement class `Computer`.

3.2. Test class `Computer` in a driver program that does the following:

- instantiates an object of class `Computer`, with the following details:

```
speed: 1600
main memory: 4
hard disk: 16
```

- use the accessor functions to display the specifications of the instantiated object on the console
- display the specifications of the instantiated object on the console with the member function `display_specs()`.

3.3. Derive and implement a class `Laptop` from class `Computer`. This class has additional member variables, `battery_time` and `weight`. Class `Laptop` also has an overloaded constructor and member functions, `get_battime()` and `get_weight()` to return member variables `battery_time` and `weight` respectively. The class `Laptop` should override function `display_specs()` in order to display the member variables of `Laptop`.

Implement the overloaded constructor for the class `Laptop` by invoking the base class constructor.

3.4. Test class `Laptop` in a driver program that does the following:

- instantiates an object of class `Laptop`, with the following details:
 - speed: 1333
 - main memory: 2
 - hard disk: 8
 - battery time: 8
 - weight: 1.25
- use the accessor functions to display the specifications of the instantiated object on the console.

4. Question 2 [20]

Design and implement a C++ class called **Product** that handles information regarding product prices, warranty plans, and discounts offered when purchasing a specific product. Think of all the operations you would want to perform with such a class and write corresponding member functions for your **Product** class. Your class declaration should be well-documented so that users understand how to use it.

Write a main program that does the following:

- Declare an array to hold information for different types of products. The elements of the array must be of type **Product**.
- You must have member variables to store:
 - The price of a product
 - The value (in Rands) of the warranty plan
 - The type/category of the product
 - Discount amount or rate
 - Any other relevant details
- Initialise the array with appropriate product information.

The criterion for receiving a special discount is that the buyer agrees to purchase a **warranty plan**.

- Determine the total price of each product by:
 - Deducting the applicable discount amount from the base price
 - Adding the value of the warranty plan
 - This gives the final price of the product
- Display the following for each product:
 - Price including the warranty plan (without discount)

- Price including the warranty plan (with discount applied)

Enrichment Exercises (Optional Practice)

(a) Convert your Product class into an Abstract Data Type (ADT) by:

Separating the interface (.h file) and implementation (.cpp file)

Using separate compilation to compile the implementation independently from the application file

(b) Modify the application program to use a vector instead of an array.

No changes should be required in the class interface or implementation.

5. Question 3 [28]

- 5.1. What is a pointer? (1)
- 5.2. What is a dereferencing operator? (1)
- 5.3. What is the difference between the assignment statement $p1=p2$; and $*p1 = *p2$; (1)
- 5.4. What is a dynamic variable? (1)
- 5.5. What is the purpose of the new operator? (1)
- 5.6. What is the purpose of the delete operator? (1)
- 5.7. What is the freestore (also called the heap)? (1)
- 5.8. What is a dynamic array? (1)
- 5.9. What is the relationship between pointers and arrays? (1)
- 5.10. Write statements to do the following:
 - i. Define two pointer types p1 and p2 for pointer variables that contain pointers to int variables (1)
 - ii. Declare two int variables a and b and initialize b to 100. (1)
 - iii. Let p1 point to b. (1)
 - iv. Dynamically allocate an integer variable and store its address in p2. (1)
 - v. Divide the value that p1 is pointing to by 4 and assign the result to a. (1)
 - vi. Assign the result of b divided by a to the variable that p2 is pointing to. (1)
 - vii. Define an int pointer p3. (1)
 - viii. Let p3 point to a. (1)
 - ix. Assign the value of the variable that p2 is pointing to, to the variable that p3 is pointing to. (1)
 - x. Add 6 to variable a. (1)
 - xi. Assign twice the value of the variable that p3 is pointing to, to the variable that p1 is pointing to (1)

- xii. Let p1 point to the same variable as p2. (1)
- xiii. What is the value of the variable that p1 is pointing to at this stage? (*Hint: Add statements (i) – (xiii) in a C++ program and display the value of the variable in question*). (1)
- xiv. Free the memory allocated to the variables that p1, p2 and p3 are pointing to. (1)

5.11. Consider the following code fragment which is assumed to be embedded in a complete and correct C++ program:

```

1.   int *p1, *p2, x = 20, y = 10;
2.   p1 = &x;
3.   *p2 = x - 10;
4.   *p1 = y + 5;
5.   int *p3 = new int;
6.   *p3 = *p1;
7.   p2 = p3;
8.   cout << *p1 << ' ' << *p2 << ' ' << *p3 << endl;
9.   //release p2

```

- 5.11.1. The statement in line 3 is a risky statement. Why? (1)
- 5.11.2. What is the purpose of the statement in line 5? (1)
- 5.11.3. What is the purpose of the statement in line 2? (1)
- 5.11.4. What is the purpose of the * in line 1? (1)
- 5.11.5. What is the purpose of the * in line 4? (1)
- 5.11.6. Assuming that we do not execute line 3, what is the output after line 8? (1)
- 5.11.7. Write down a statement for line 9 to release p2 to the freestore. (1)

6. Question 4 [30]

Use separate compilation to define a class called **Gradebook** that represents a student's academic performance record. This class contains three member variables:

- **studentName**, a string that holds the name of the student
- **score**, an integer variable that holds the student's final mark
- **gradeLevel**, an integer that represents the academic level/year achieved (e.g., level completed or grade passed)

Required Member Functions:

The class should contain the following member functions:

- A **default constructor** that initializes:
 - `studentName` to an empty string

- o `score` and `gradeLevel` to 0
- An **overloaded constructor** that accepts a student's details and sets:
 - o `studentName`, `score`, and `gradeLevel` to specified values
- A **destructor** that does not perform any action
- An **accessor function** `get_score()` to return the value stored in a student's score
- An **overloaded operator** `>=` to compare one student's performance with another, to determine if one student has performed as well as or better than another

This operator is implemented as a **friend function** with the following prototype:

```
bool operator>=(const Gradebook & student1, const Gradebook & student2);
```

This function returns:

- **true** if `student1` has at least the same score and grade level as `student2`
- **false** otherwise
- An **overloaded extraction operator** (`>>`) implemented as a friend function
 - o Used to input student records from a file
- An **overloaded insertion operator** (`<<`) implemented as a friend function
 - o Displays a student's name, score, and grade level

Application Requirements:

- 6.1. Create the header file **gradebook.h** that contains the **Gradebook class specification**.
- 6.2. Create the implementation file for the **Gradebook class**, including all member functions and friend functions.
- 6.3. Demonstrate the class in an application program (`main()`) that:
 - Lists all students who have **equalled or outperformed a benchmark student**
 - Prompt the user to enter:
 - o Score
 - o Grade level
 - Use the **overloaded constructor** to initialise a Gradebook object called **benchmark_student**
 - o Set the name to an empty string
 - Student records are stored in a file called **Students.txt**
 - Use a **while loop** to:
 - o Read student records from the file using `>>`

- Compare each student with `benchmark_student` using the overloaded `>=` operator
- Display all students who meet or exceed the benchmark

6.4. Why is no code required for the destructor?

6.5. Why are the operators `>=`, `>>` and `<<` implemented as friend functions?

Sample of Student.txt data

```
Alice 75 2  
Sizwe 60 1  
Charlie 85 3  
David 75 2  
Eve 90 3  
Thandi 50 1  
Grace 80 2  
Mpilo 85 3  
Diana 45 1  
Lesedi 90 3
```